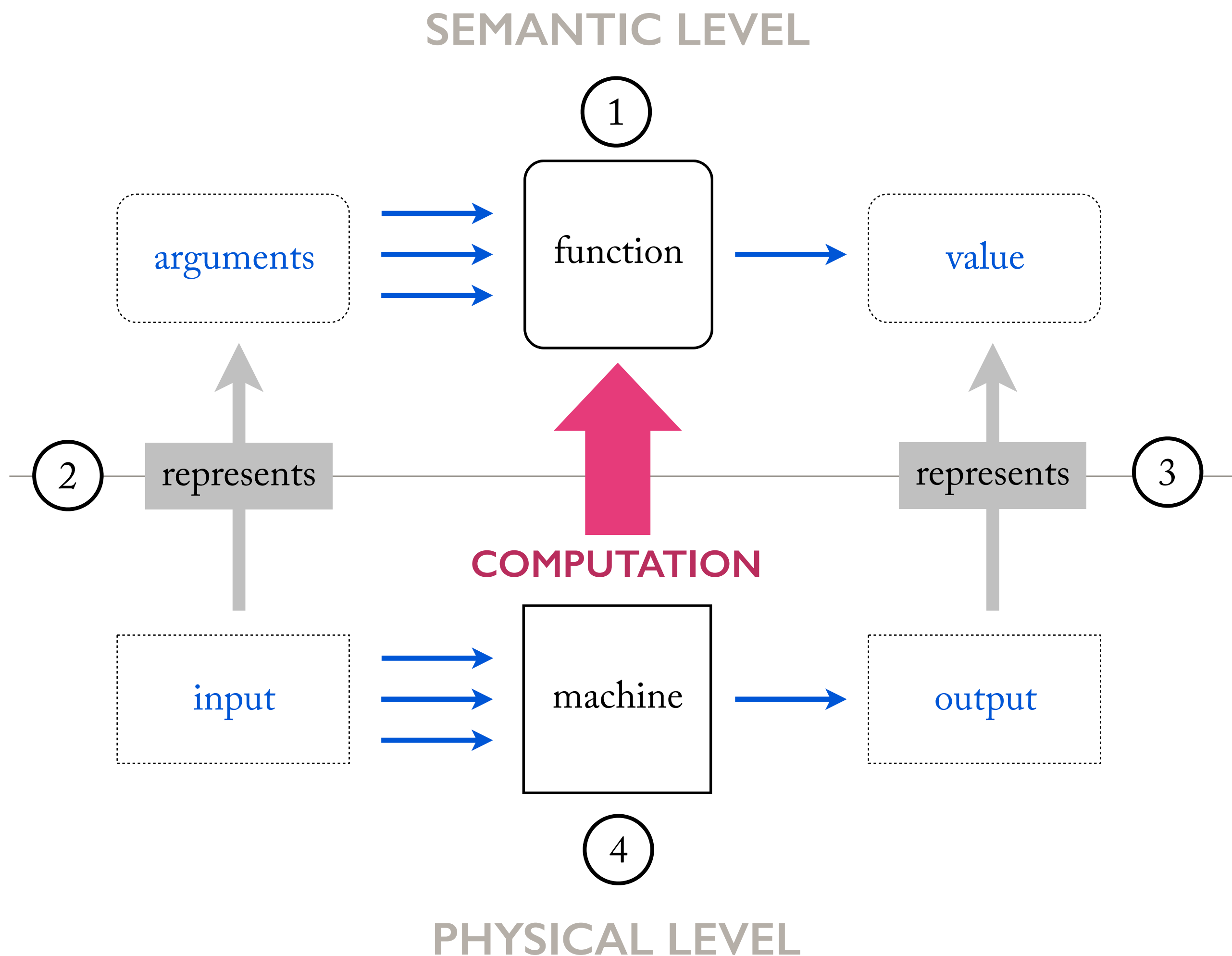


6. Defining Computation

Integrate more of Crane reading.
Distinguish computing vs.
implementing a function.

The structure of computation

Computation is a relation between a **machine** and a **function** that is brought about when the machine is designed to physically **mirror** or **simulate** the abstract argue-value profile of the function. In this section, we will define this relationship formally.



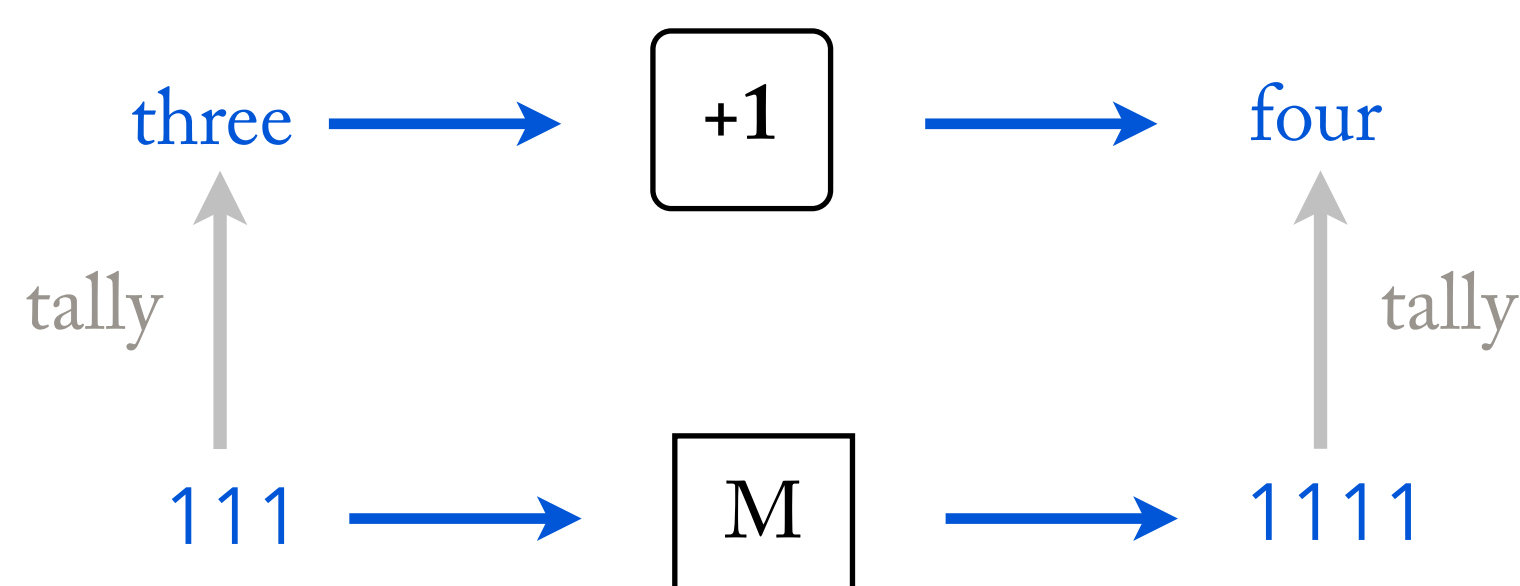
Definition of computation

Machine M **computes** function f under system R if and only if:

For every possible argument x for function f :

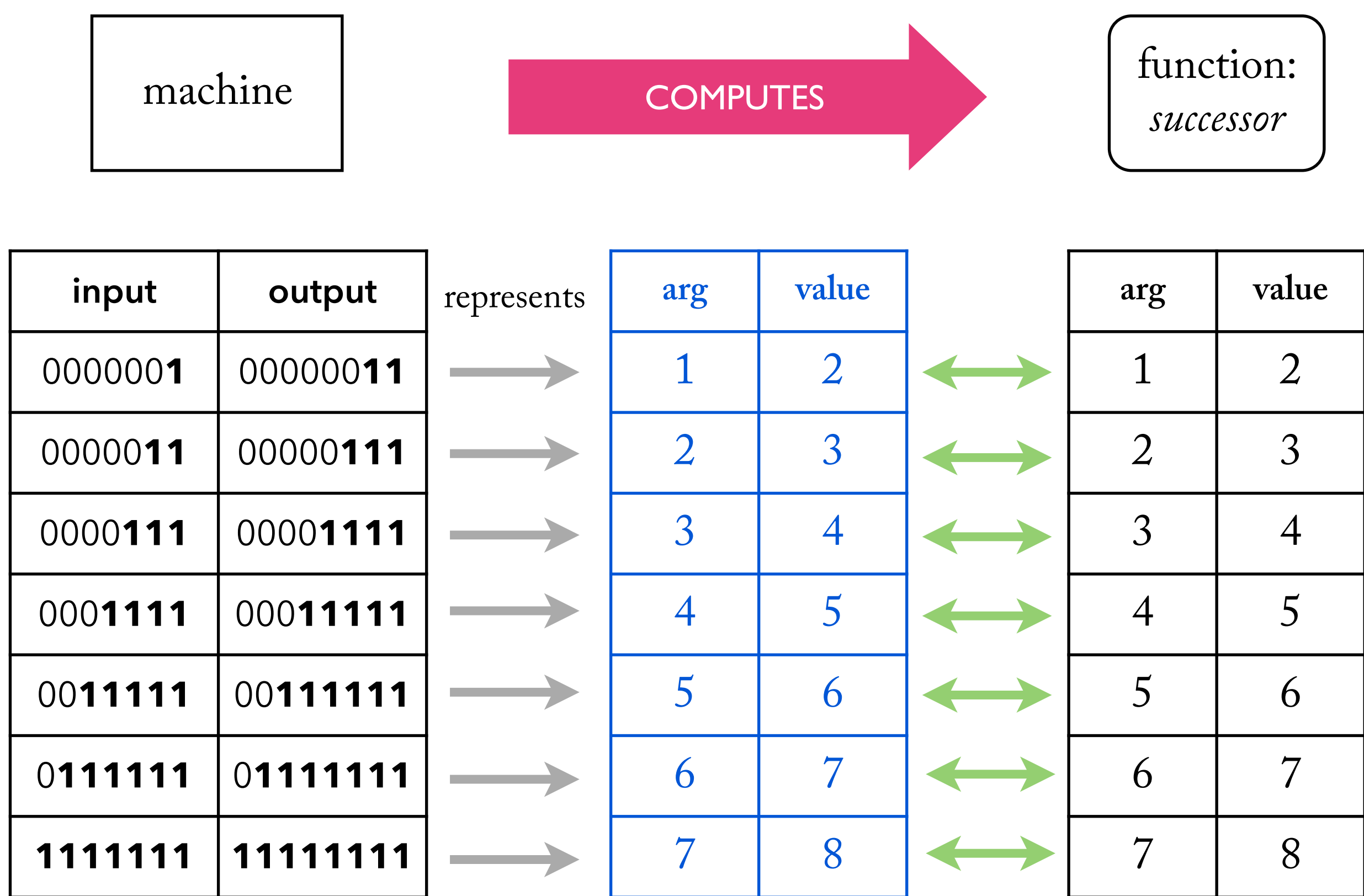
- (1) there is a value y such that $f(x)=y$;
- (2) there is a symbol string S_1 such that S_1 represents x under R ;
- (3) there is a symbol string S_2 such that S_2 represents y under R ;
- (4) if M takes S_1 as input, then M produces S_2 as output.

For at least one argument, the machine at right computes successor (+1) under the tally representation system.

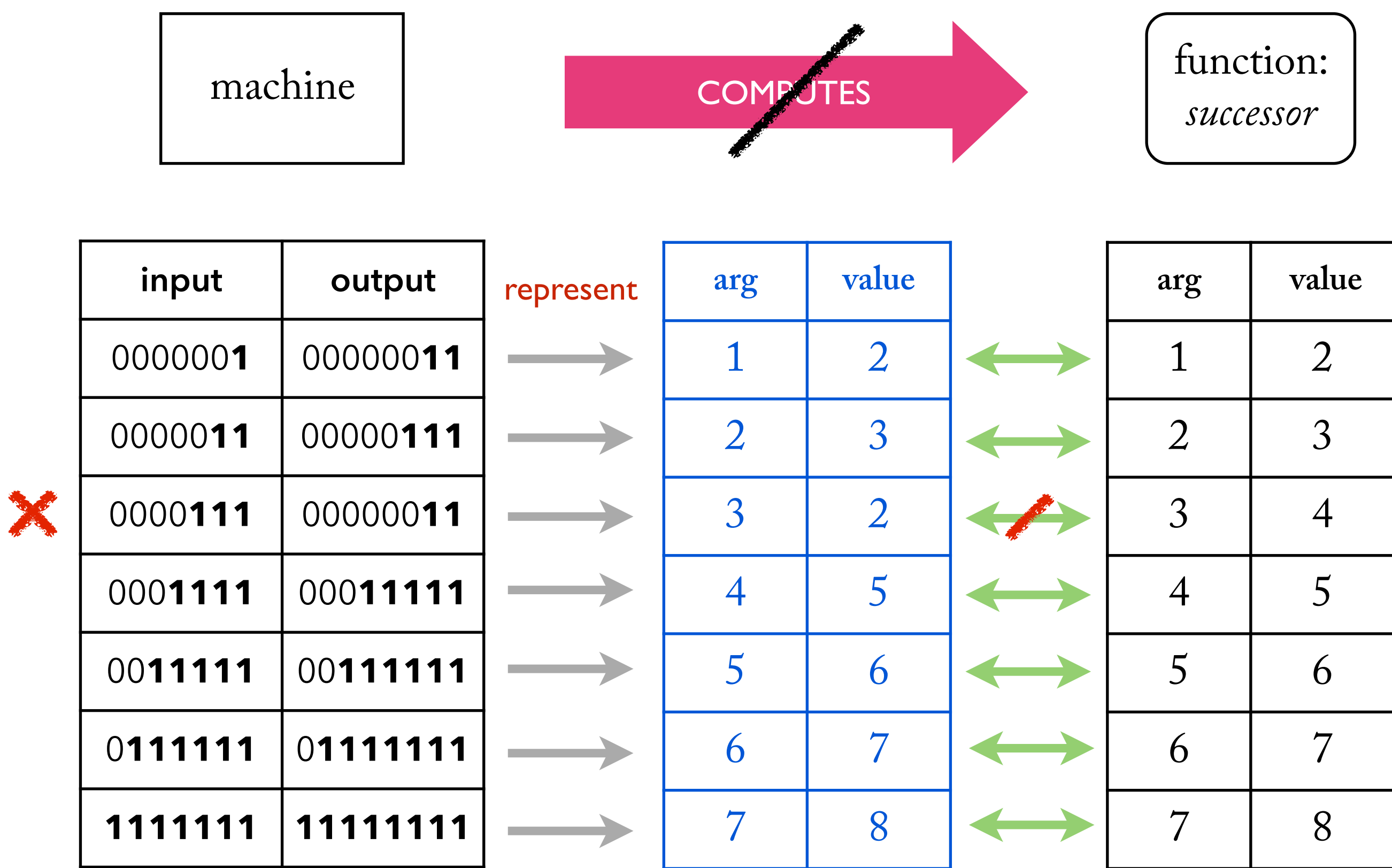


Examples of computation

The following machine successfully computes the successor function, under the tally representation system. The machine’s input-output profile, when interpreted in tally, perfectly matches the argument-value profile of the successor function (so far).



The following machine does *not* computes the successor function under the tally representation system. The machine’s input-output profile, when interpreted in tally, does *not* perfectly match the argument-value profile of the successor function.



Bounded Functions

A **bounded** function is a function which is defined for only a finite range of arguments. To name a bounded function, we write:

$f[0-n]$ e.g. “*successor*[0-3]”

Note: this names a function that is only defined for the *arguments* 0 through n (*not* the values). The same notation applies whether the function is 1-place, 2-place, or more. For example the following two tables are equivalent to the named function in each case.

successor[0-2]

arg	val
0	1
1	2
2	3

addition[0-1]

arg1	arg2	val
0	0	0
0	1	1
1	0	1
1	1	2

A machine **computes a bounded function** when it computes that function’s values for all *defined* arguments. The values it computes for undefined arguments are irrelevant for now.

Notation for functionally describing computers

We will often want to refer compactly to a computer in terms of what function it computes, and what system of representation it computes it under. This is actually a deep and important form of **functional abstraction**, as we’ll see in a few weeks.

I will say that a machine computes

$f[0-n]$ S

where “ $f[0-n]$ ” is a (bounded) function, and S is a representational system. Here I will normally use **T** to stand for tally, and **B** to stand for binary. So I will often ask you to solve problems such as:

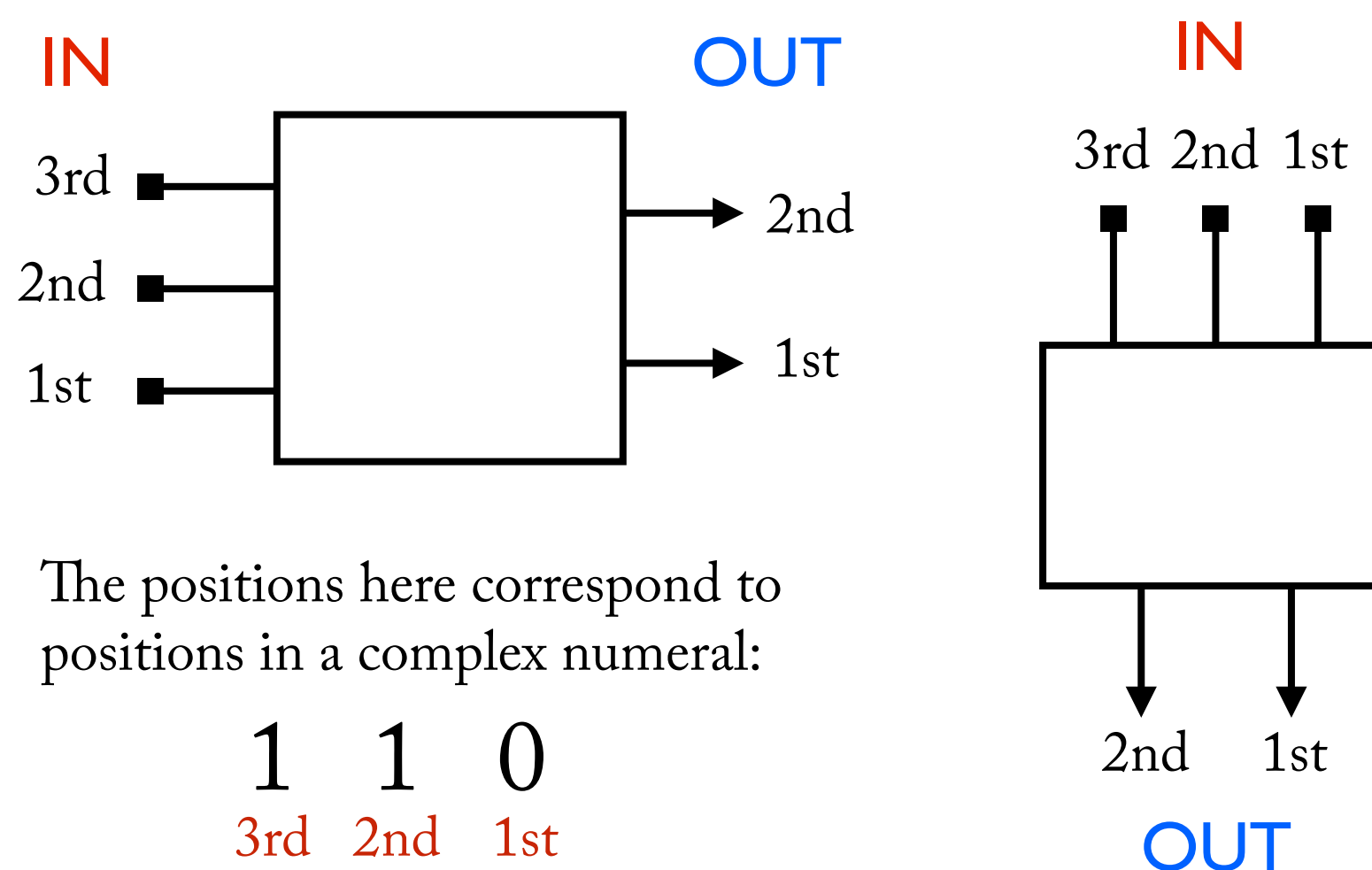
Design a machine which computes $+1[0-9]$ B.

Design a machine which computes $2(x+3)$ T.

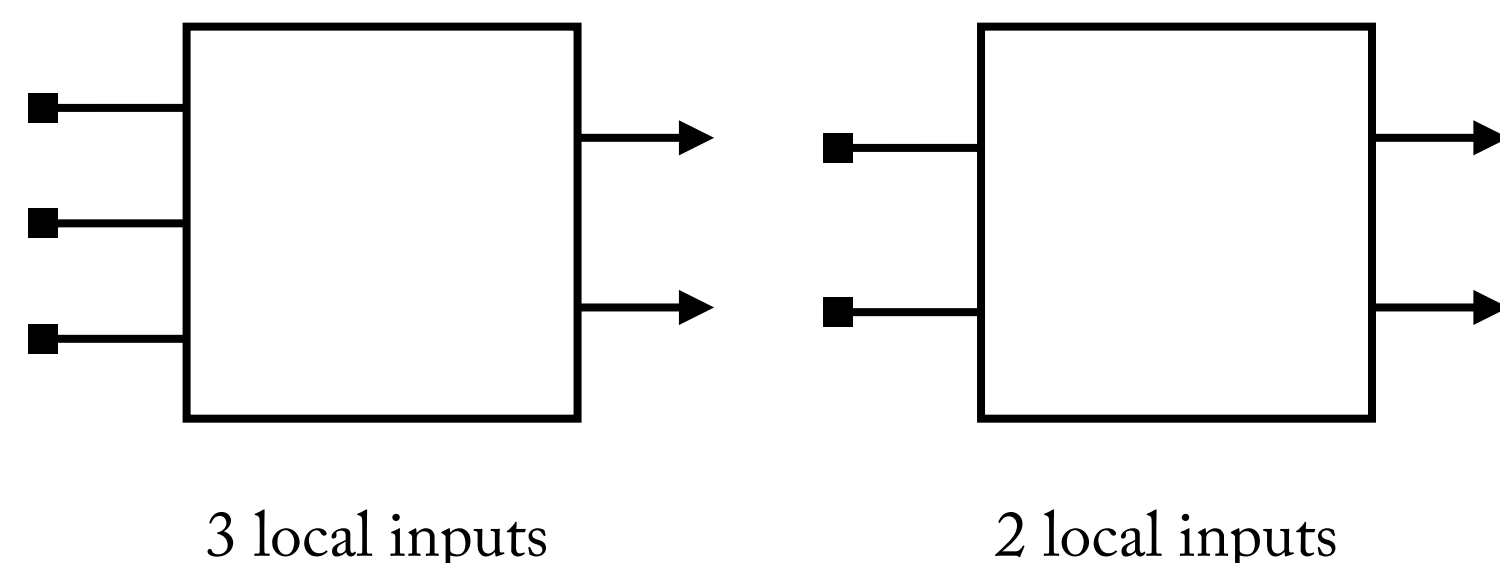
I’ll sometimes omit variables in the definition of the function, if context makes their role clear, as in the first machine described above.

Inputs and Outputs

Input and output. A CC can have any number of inputs and any number of outputs. All the inputs are read at the same time (**synchronously**) and all the outputs are produced at the same time. By convention, the inputs and outputs are ordered from top-to-bottom, or from right-to left (following Arabic).

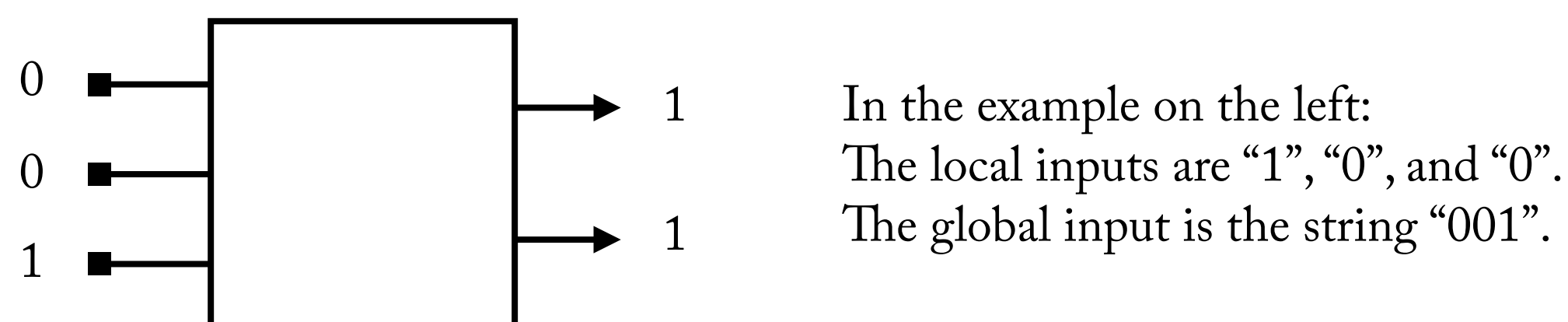


Local input. The **local** inputs to a circuit correspond to the number of input-wires the machine has. Each local input receives one symbol at a given time. A single symbol is called an **atomic** symbol.



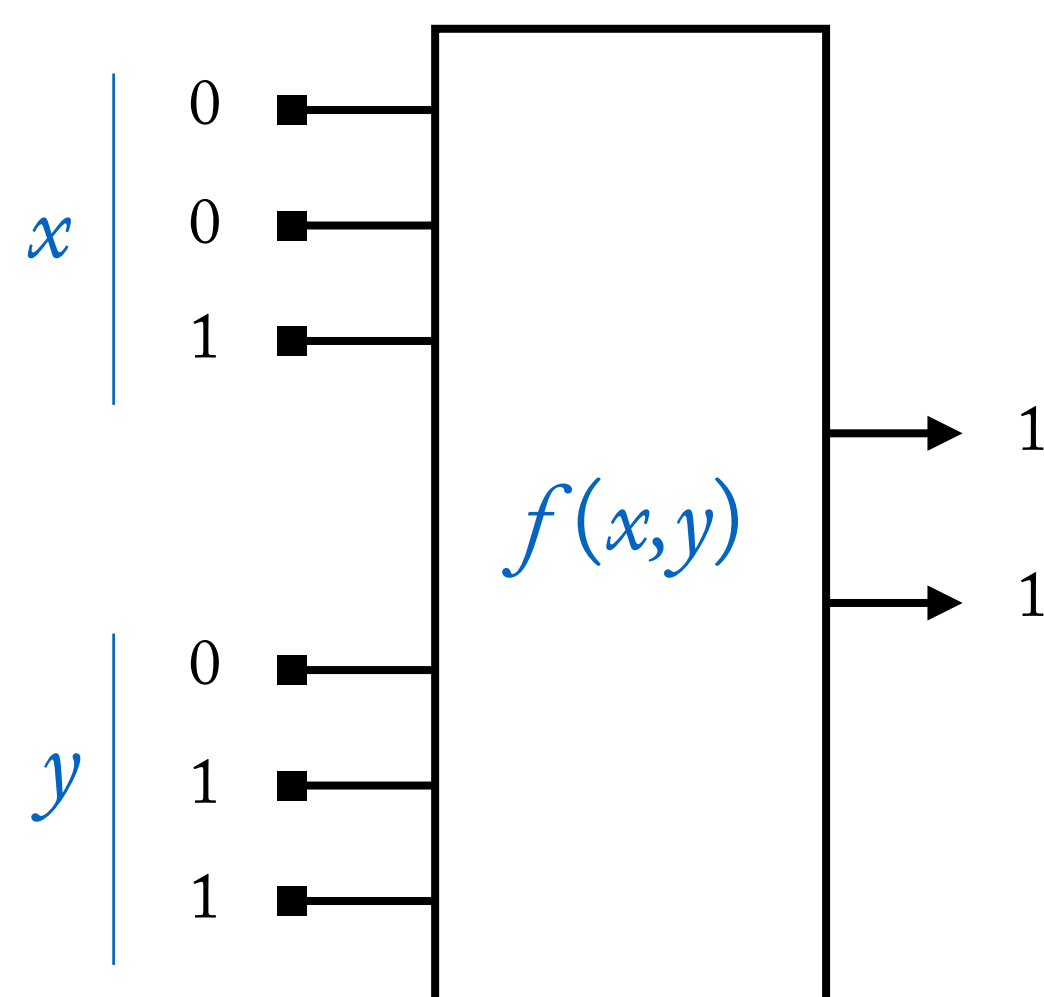
Global input. The **global** input to the machine is a collection of its local inputs. Where each local input carries an atomic symbol, a global input is the aggregate or **string** of atomic symbols. This string itself is called a **complex** symbol. The **order of local inputs** within a global input corresponds to the **order of symbols** in the input string.

In a CC, the global input is read at one time, by reading each local input synchronically. Later we'll look at circuits where the global input is read over time, **diachronically**.



Multiple global inputs. The number of global inputs corresponds to the number of arguments in the function being computed. Each global input represents one argument.

When designing a machine that computes a function with multiple argument places, indicate clearly which global input goes with which argument. The ordering convention again is top-to-bottom, or right-to-left.



Computation as a Three-part Relation

Computation is a relation between a **machine** M , a **system of representation** R , and a **function** f .

Given any two, you can solve for the third.

$$M + R = f?$$

$$M + f = R?$$

$$f + R = M?$$

The machine and representational system together determined the function computed. But they are themselves logically independent from one another. This means that...

The same M can compute different functions f and f^* relative to different systems of representation R and R^* .

The same function f can be computed by different machines M and M^* relative to different systems of representation R and R^* .

This demonstrates the importance of representational system for designing a computer. One must get the circuitry of the machine right, but it must also be tuned to the right representational system.

In-class Problems

1. Suppose M1 computes $x+1[0-4]$ T
 - a. How many local inputs and outputs must M1 have?
How many global inputs and outputs does M1 have?
 - b. Make a global input-output table for M1. (Restrict your table to possible inputs and outputs.)
 - c. Design a circuit for M1.
2. M2 computes $x+1[0-5]$ T. Design a circuit.
3. M3 computes $x+1[0-1]$ B. M3 has 1 local input and 2 local outputs.
Make a global input-output table. Design a circuit.
4. M4 computes $x+1[0-2]$ B. M3 has 2 local inputs and 2 local outputs.
Make a global input-output table. Design a circuit.
5. M5 computes $x+1[0-3]$ B. Make a global input-output table. Design a circuit.

Tip: for all these problems, turn your circuit 90° so the inputs and outputs are vertically aligned, the same way you would set up a long addition problem.